

AIS Graph Analytics

Архитектура, эксплуатация, CI/CD и документация релизов

Konstantin Lofichenko

Октябрь 2026

Содержание

1. Обзор платформы	1
2. Архитектура	2
3. Docker runtime	2
4. Установка с чистого состояния	4
5. Модель данных ClickHouse	5
6. dbt	5
7. Обработка historical AIS	6
8. Ежедневная оркестрация Airflow	6
9. AI enrichment	7
10. Neo4j и графовая аналитика	7
11. Metabase	8
12. CI	8
13. Release workflow	8
14. Опубликованные containers	9
15. Проверка и эксплуатация	9
16. Важные правила качества данных	9
17. Известное ограничение: фрагментация port visit на границе полуночи	10
18. Troubleshooting	10
19. Безопасная остановка и cleanup	10
20. Карта документации репозитория	11
21. Текущее состояние и roadmap	11

Этот документ объединяет актуальное описание архитектуры, настройки, эксплуатационных правил, CI/CD, процесса релизов и известных ограничений портфолио-проекта AIS Graph Analytics.

Документ является эксплуатационным и архитектурным дополнением к корневому README.md репозитория. Он отражает текущее состояние платформы после завершения CI/CD и публикации релизов.

1. Обзор платформы

AIS Graph Analytics объединяет два режима загрузки данных:

- **Live:** BarentsWatch AIS -> Python producer -> Kafka.
- **Historical:** H AIS GeoParquet -> Airflow -> ClickHouse.

Оба потока формируют каноническую историю AIS в ClickHouse, которую используют аналитические модели dbt. Neo4j хранит графовые структуры и текущее состояние, GDS рассчитывает графовые метрики, OpenAI обогащает выбранные дневные аномалии судов, а Metabase предоставляет пользовательскую аналитику.

Архитектура намеренно разделяет зоны ответственности:

Задача	Система учета / основной движок
История исходных событий	ClickHouse
Аналитические преобразования	dbt on ClickHouse
Текущее графовое состояние судов	Neo4j
Графовая аналитика портов и сообществ	Neo4j GDS + ClickHouse snapshots
Транспорт live-данных	Kafka
Пакетная / ежедневная оркестрация	Airflow
AI-интерпретация	Внешний pipeline обогащения OpenAI
Визуализация	Metabase
CI/CD	GitHub Actions + GHCR

2. Архитектура

2.1 Live path

BarentsWatch

- > producers/ais
- > Kafka topic ais.positions
 - > ClickHouse Kafka Engine / materialized view -> raw.ais_positions
 - > Kafka Connect Neo4j sink -> current Vessel nodes

Producer использует MMSI как ключ Kafka и сохраняет исходные поля AIS. ClickHouse хранит историю событий, а Neo4j sink обновляет текущее состояние судна по MMSI.

2.2 Historical path

HAIS GeoParquet files

- > Airflow historical ingestion
- > raw.hais_positions
- > dbt staging normalization
- > canonical load macro
- > raw.ais_positions

Landing-слой HAIS намеренно сохраняет данные максимально близко к источнику. Нормализация выполняется до загрузки в каноническую таблицу AIS.

2.3 Аналитический / графовый path

raw.ais_positions

- > dbt vessel models
- > daily features
- > deterministic anomalies
- > AI enrichment history
- > daily enriched vessel view

raw.ais_positions

- > port visit detection
- > Neo4j port graph
- > PageRank + Louvain
- > ClickHouse graph snapshots
- > Metabase

3. Docker runtime

3.1 Compose profiles

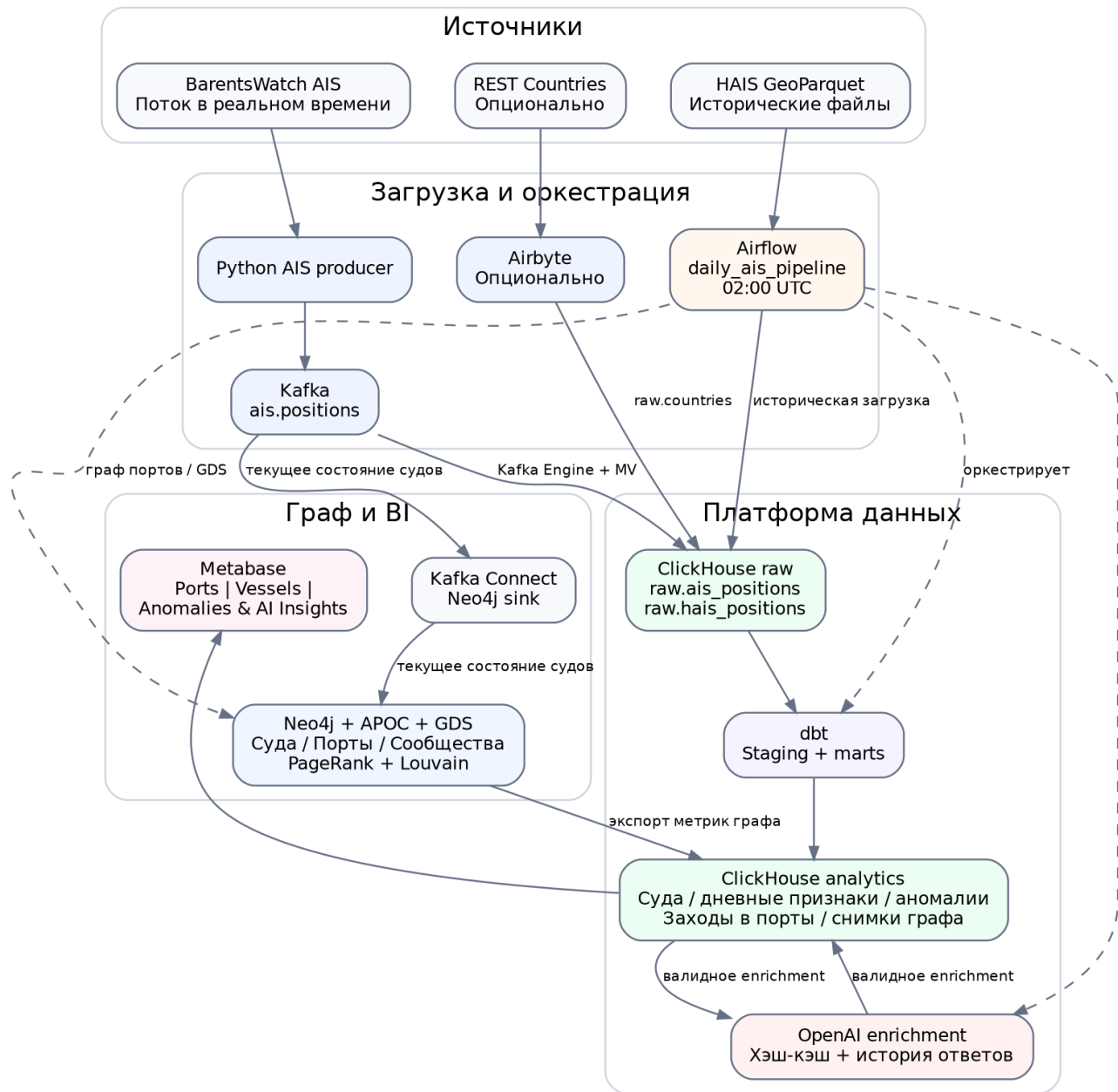


Рис. 1: Архитектура AIS Graph Analytics

Profile	Основной service	Назначение
none	Kafka, Kafka Connect, ksqldb, Kafbat UI, ClickHouse, Neo4j	Основной runtime
live	AIS producer	Непрерывная загрузка BarentsWatch
batch	Airflow	Историческая загрузка и ежедневная аналитика
analytics	Metabase	BI / dashboarding

3.2 Собственные образы проекта

Release workflow публикует три образа в GHCR:

```
ghcr.io/konstantinlofichenko/ais-graph-analytics-airflow
ghcr.io/konstantinlofichenko/ais-graph-analytics-producer
ghcr.io/konstantinlofichenko/ais-graph-analytics-neo4j
```

Образ Neo4j кастомизирован, потому что проект добавляет поддержку ClickHouse JDBC поверх upstream-образа Neo4j. Временные имена образов *-ci являются тестовыми артефактами и не публикуются как release packages.

3.3 Сторонние образы

Kafka, ClickHouse, Metabase, ksqldb, Kafbat UI и другие инфраструктурные компоненты используются напрямую из их официальных registries. Они намеренно не перепубликуются как собственные packages проекта.

4. Установка с чистого состояния

4.1 Репозиторий и окружение

```
git clone https://github.com/KonstantinLofichenko/ais-graph-analytics.git
cd ais-graph-analytics
cp .env.example .env
mkdir -p data/hais
```

Настройте локальные secrets в .env. Как минимум задайте учетные данные ClickHouse и Neo4j. Учетные данные BarentsWatch нужны только при использовании live ingestion.

Никогда не коммитьте .env, пароли connector и развернутые конфигурации, содержащие secrets.

4.2 Локальная конфигурация Kafka Connect

```
cp neo4j/connectors/ais-sink.example.json neo4j/connectors/ais-sink.json
```

Внутри Docker используйте:

```
bolt://neo4j:7687
```

localhost из Kafka Connect указывает на сам контейнер Kafka Connect, поэтому для доступа к Neo4j это неверный адрес.

4.3 Запуск основных services

```
docker compose up -d --wait --wait-timeout 300
docker compose ps
```

4.4 Bootstrap

`./scripts/bootstrap.sh`

Bootstrap script:

1. Проверяет локальную конфигурацию connector.
2. Создает или проверяет Kafka topic `ais.positions`.
3. Применяет аддитивные миграции ClickHouse 003-010.
4. Применяет constraints Neo4j.
5. Создает или обновляет Neo4j Kafka sink connector.

Миграция `001_replacing_ais_positions.sql` предназначена для уже существующих установок и намеренно не входит в fresh bootstrap.

5. Модель данных ClickHouse

5.1 Канонический raw AIS

`raw.ais_positions` является канонической долговременной таблицей событий. Она оптимизирована для доступа по судну и времени и использует месячные partition по event time.

Важные правила:

- основной аналитический ключ доступа: `(mmsi, msgtime)`;
- version column: `ingested_at`;
- engine: `ReplacingMergeTree(ingested_at)`;
- для логической проверки используйте FINAL, когда важна сходимость заменяемых версий;
- live и historical источники могут пересекаться, поэтому перед крупными backfill следует проверять полноту атрибутов источников.

5.2 Historical HAIS

`raw.hais_positions` сохраняет исходные значения HAIS для аудита. dbt staging нормализует детали источника перед канонической загрузкой.

5.3 История port visits и графа

`analytics.port_visits` и `analytics.port_graph_metrics` используют replacement/tombstone semantics. Устаревшие логические записи помечаются более новой deleted-версией вместо физического удаления.

Downstream-потребители должны работать с логически активным состоянием (FINAL плюс фильтр active/deleted там, где он требуется).

6. dbt

6.1 Окружение

```
python3.11 -m venv .venv-dbt
source .venv-dbt/bin/activate
python -m pip install -r dbt/requirements.txt
cp dbt/profiles.yml.example dbt/profiles.yml
export DBT_PROFILES_DIR="$PWD/dbt"
export CLICKHOUSE_USER=default
read -r -s -p 'ClickHouse password: ' DBT_ENV_SECRET_CLICKHOUSE_PASSWORD
printf '\n'
export DBT_ENV_SECRET_CLICKHOUSE_PASSWORD
```

```
dbt deps --project-dir dbt --profiles-dir dbt
```

```
dbt parse --project-dir dbt --profiles-dir dbt
dbt debug --project-dir dbt --profiles-dir dbt
```

Текущие закрепленные версии: dbt Core 1.12.5 и dbt-clickhouse 1.10.3.

6.2 Основные модели

Model	Назначение
vessels	Идентификация судна и текущее аналитическое состояние
vessel_daily_features	Дневные признаки движения и качества данных
vessel_daily_anomalies	Детерминированный выбор аномалий
vessel_daily_enriched	Дневные признаки + актуальное валидное AI enrichment
current_port_visits	Логически активные текущие port visits
port_graph_metrics_enriched	Метрики графа с reference dimensions
port_graph_communities	Таблица отчетности по сообществам

Статические соответствия AIS ship type и navigation status реализованы как dbt seeds, а не как повторяющийся ad-hoc SQL в разных моделях.

7. Обработка historical AIS

Historical-файлы вручную помещаются в data/hais с именами вида:

hais_2026-09-01.snappy.parquet

Historical Airflow ingestion сохраняет audit/retry status. После успешной загрузки источника dbt staging и canonical-load макро заполняют raw.ais_positions.

Правила дат:

Операция	Начало	Конец
Загрузка H AIS-файла	Включительно	Включительно
Каноническая загрузка dbt	Включительно	Исключительно, UTC midnight
Daily analytics	Activity date / начало UTC window	Следующая граница UTC

Избегайте неконтролируемых перекрывающихся повторных загрузок: каноническая таблица может свести duplicate versions, но backfill все равно способен заменить атрибуты, например имя или тип судна, если historical-источник менее полный, чем live-источник.

8. Ежедневная оркестрация Airflow

Главный DAG:

daily_ais_pipeline

Расписание:

02:00 UTC daily

Для детерминированных rerun/backfill можно передать явный dag_run.conf.activity_date.

Daily orchestration координирует аналитическую цепочку, а не рассматривает dbt, AI, graph и dashboard data как независимые ручные задания.

Концептуально:

canonical AIS ready
-> dbt staging/marts
-> anomaly candidates
-> AI enrichment
-> port visits
-> Neo4j sync / graph analytics
-> graph snapshot export
-> dashboard-ready data

Historical backfill следует выполнять последовательно там, где задействованы обновления graph/current state.

9. AI enrichment

9.1 Назначение

AI используется для краткого описания выбранных аномальных vessel-days, а не для замены детерминированного feature engineering или выбора аномалий.

9.2 Cache и история

analytics.vessel_ai_enrichment сохраняет историю ответов, включая model, prompt version, точный input hash, token counts, response ID и время создания.

Lookup требует точного совпадения ключевых параметров входа, включая date/model/prompt/input hash. Это предотвращает показ устаревшего enrichment после изменения исходных features и одновременно сохраняет исторические ответы для аудита.

9.3 Ограничение количества вызовов

AI_ENRICHMENT_LIMIT ограничивает только **новые** API calls после исключения cache hits. CI не выполняет live-вызовы OpenAI.

10. Neo4j и графовая аналитика

10.1 Текущее состояние и история

Neo4j представляет текущее состояние графа. ClickHouse хранит версионированные по времени snapshots графа для исторического анализа.

10.2 Основные relationships

```
(Vessel) - [:VISITED] -> (Port)
(Port) - [:CONNECTED_TO] -> (Port)
(Port) - [:MEMBER_OF] -> (Community)
```

10.3 GDS

Graph workflow включает:

- PageRank для оценки важности портов;
- Louvain для community detection;
- детерминированные названия и labels сообществ на основе representative ports;
- экспорт метрик графа в ClickHouse.

Отчетность по сообществам использует понятные человеку названия и при этом сохраняет технический community ID для traceability.

11. Metabase

Главный dashboard содержит три логических tabs:

1. **Ports**
2. **Vessels**
3. **Anomalies & AI Insights**

Проект также включает export/import helpers, поэтому конфигурацию Metabase можно хранить как проектные assets, близкие к подходу infrastructure-as-code, а не только как ручное состояние UI.

Dashboard date filters используют бизнес-семантику activity_date вместо технических run IDs там, где это возможно.

12. CI

CI workflow намеренно изолирован от внешних services и локального состояния разработчика.

12.1 Jobs

Job	Что проверяется
Shell and Compose Pipeline tests	Синтаксис shell и все Compose profiles Airflow image + изолированные DAG/pipeline tests
Producer dbt + ClickHouse	Python producer tests + сборка image Чистый ClickHouse, init, migrations, CI fixture, seeds, выбранный graph моделей/tests
Kafka + Neo4j	Kafka topic/produce/consume + запуск custom Neo4j/constraints/write/read

12.2 CI source fixture

Интеграционный тест dbt создает детерминированный fixture raw.countries только для CI, потому что в production raw.countries обычно принадлежит опциональной загрузке Airbyte. Fixture должен оставаться только в CI и не должен добавляться в обычную production initialization.

12.3 Внешние вызовы, намеренно исключенные из CI

CI не зависит от:

- BarentsWatch;
- OpenAI;
- Airbyte control plane;
- работающего локального Mac environment;
- production Metabase.

Благодаря этому pull-request checks остаются детерминированными и воспроизводимыми.

13. Release workflow

Stable semantic tag формата vMAJOR.MINOR.PATCH запускает release automation.

Пример:

```
git tag -a v1.0.3 -m "Release v1.0.3"
git push origin v1.0.3
```

Workflow:

1. Проверяет формат semantic version.
2. Проверяет, что release commit входит в main.
3. Запускает reusable CI workflow.
4. Собирает Airflow, producer и custom Neo4j images.
5. Публикует versioned images в GHCR.
6. Создает GitHub Release с автоматически сгенерированными notes.

Stable image tags:

MAJOR.MINOR.PATCH
MAJOR.MINOR
MAJOR
latest

Не перемещайте уже опубликованный release tag. Используйте новую patch/minor/major version.

14. Опубликованные containers

```
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-airflow:latest
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-producer:latest
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-neo4j:latest
```

Публикуются только собственные images проекта. Сторонние images продолжают использоваться из upstream registries.

15. Проверка и эксплуатация

Полезные проверки проекта:

```
./scripts/status.sh
docker compose ps
docker compose logs --tail 100 airflow
docker compose logs --tail 100 ais-producer
docker compose logs --tail 100 neo4j
```

Проверка Kafka topics:

```
docker exec ais-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server ais-kafka:29092 --list
```

При проверке логического состояния ClickHouse следует использовать FINAL, когда важны replacement semantics ReplacingMergeTree.

16. Важные правила качества данных

- Полнота ship type/name зависит от источника; source NULL не следует молча заменять выдуманной идентичностью.
- AIS navigation status может быть шумным; это вспомогательный сигнал, а не окончательная классификация движения.
- Speed metrics исключают неправдоподобные значения с помощью настроенного validity threshold.
- Stationary percentage основан на наблюдениях, а не на непрерывной интерполяции по elapsed time.
- AI summaries описывают engineered inputs, которые получают; они не доказывают поведение судна между AIS observations.
- Graph community IDs являются техническими identifiers; labels/names предназначены для пользователя.

17. Известное ограничение: фрагментация port visit на границе полуночи

Port-visit detection обрабатывает ограниченные UTC windows. Если одно физическое пребывание в порту пересекает полночь, оно может быть представлено как отдельные daily visit fragments, потому что state не сохраняется между daily windows.

Влияние:

- возможный завышенный подсчет visits;
- фрагментация duration;
- небольшие downstream-различия в graph connections/metrics.

Production-grade альтернативы: persistent detector state либо непрерывное определение физических visits с последующей daily projection.

18. Troubleshooting

Конфликт имени container

```
docker ps -a
```

Удаляйте только устаревшие AIS test/manual containers. Не удаляйте вслепую project volumes.

Kafka Connect не может подключиться к Neo4j

Внутри Docker используйте bolt://neo4j:7687, а не localhost.

Healthchecks ksqldb или Kafka Connect

Используемые в проекте Confluent images не зависят от curl для этих healthchecks; Compose configuration использует TCP checks там, где это необходимо.

Custom image Neo4j

Собственный Neo4j image проекта включает ClickHouse JDBC JAR и проверяется в CI с тем plugin environment, который используется обычным Compose service.

dbt не может подключиться

Проверьте:

- активировано ли virtual environment dbt;
- указывает ли DBT_PROFILES_DIR на каталог dbt проекта;
- совпадает ли DBT_ENV_SECRET_CLICKHOUSE_PASSWORD с локальным паролем ClickHouse;
- правильны ли host/port для тестируемого environment.

19. Безопасная остановка и cleanup

Остановить services без удаления named volumes:

```
docker compose --profile live --profile batch --profile analytics down
```

Не добавляйте -v, если данные нужно сохранить.

Временные локальные CI images можно удалить, когда они больше не нужны:

```
docker images --format '{{.Repository}}:{{.Tag}}' | grep -- '-ci'
```

Удаляйте только disposable test images, а не project runtime images и upstream dependencies, которые вы продолжаете использовать.

20. Карта документации репозитория

Path	Назначение
README.md	Обзор проекта и quick start
docs/README.md	Подробное эксплуатационное и архитектурное руководство
airflow/README.md	Настройка и оркестрация Airflow
dbt/README.md	Конфигурация dbt и transformations
pipelines/port_visits/README.md	Реализация port visits
pipelines/graph_metrics/README.md	Экспорт graph metrics
neo4j/gds/README.md	GDS workflows

21. Текущее состояние и roadmap

Текущая реализация включает:

- live + historical AIS ingestion;
- каноническую историю ClickHouse;
- аналитические модели и tests dbt;
- scheduled orchestration Airflow;
- графовую аналитику Neo4j;
- детерминированный выбор аномалий;
- AI enrichment с cache/history;
- dashboards Metabase;
- CI из пяти jobs;
- semantic GitHub Releases;
- публикацию GHCR для всех собственных runtime images проекта.

Планируемые учебные расширения, такие как Flink, MinIO, Iceberg, Trino и dbt-trino, относятся к будущей работе и не должны описываться как часть текущего deployed runtime до их реализации.